

Università degli Studi di Perugia

Dipartimento di Informatica / Ingegneria Informatica

Corso di Intelligenza Artificiale

Analisi e Digitalizzazione di Mappe
Geopolitiche Tramite Computer Vision
e Deep Learning

Studenti:

Romeo Miscio 364672
Tommaso Moschini
Federico Tabuani

Docente:

Prof.ssa Valentina Poggiani

Anno Accademico 2025/2026

Ultima modifica: 05/06/2026

Indice

1	Definizione del Dominio: Navigazione e Mappatura Territoriale	2
1.1	Architettura della Mappa e Discretizzazione	2
1.2	Logica di Movimento e Modello dei Costi	2
1.3	Classificazione e Tipologia dei Territori	3
2	Architettura del Software e Scelte Progettuali	3
3	Sviluppo del Modello ML: Riconoscimento dei Caratteri	4
3.1	Creazione del Modello: Dalla MLP alla CNN	4
3.2	Il Dataset EMNIST e la necessità del Layer di Ragionamento	5
3.3	Tentativi di Generalizzazione e Modifiche in Inferenza	5
3.4	Filtri Anti-rumore e l'Integrazione finale di SciPy	5
4	Il Modulo Digitalizzatore: Computer Vision e Pre-elaborazione	6
4.1	Evoluzione del Programma: Dalla Base alle Soglie Adattive	6
4.2	Gestione del Tratto di Penna e Rimozione Quadretti	6
5	L'Hub Master: Integrazione e Logica di Sistema	7
5.1	Classificazione tramite Analisi delle Aree	7
5.2	Topologia degli Stati e Colorazione	7

1 Definizione del Dominio: Navigazione e Mappatura Territoriale

Il presente progetto si pone l'obiettivo di modellare un sistema di navigazione per un agente operante in un ambiente discretizzato, caratterizzato da territori frazionati e costi di transito variabili.

1.1 Architettura della Mappa e Discretizzazione

L'ambiente è rappresentato come una matrice di celle, dove la granularità della griglia determina la precisione della rappresentazione geografica:

- **Rappresentazione degli Stati:** Uno stato (o regione) può coincidere con una singola cella o essere costituito da un cluster di più celle contigue per rappresentare estensioni territoriali maggiori.
- **Segmentazione Cromatica:** Al fine di distinguere topologicamente le diverse regioni, ogni stato è caratterizzato da un colore differente rispetto a quelli adiacenti. Questo garantisce una separazione netta dei confini durante la fase di analisi dell'immagine.
- **Processo di Pixellazione:** La definizione dei confini avviene attraverso una procedura di pixellazione che mappa l'input visivo sulla griglia di navigazione (inizialmente pensata come una griglia 100×100 , in seguito resa variabile pur mantenendo le dimensioni di un quadrato), definendo l'appartenenza di ogni coordinata (x, y) a uno specifico stato.

1.2 Logica di Movimento e Modello dei Costi

Il movimento dell'agente è vincolato a spostamenti tra celle adiacenti lungo le direzioni cardinali (Nord, Sud, Est, Ovest). La funzione di costo associata alle azioni di navigazione è così definita:

- **Costo Base di Spostamento:** Il transito tra due celle adiacenti ha un costo base unitario pari a 1.
- **Penalità di Confine:** L'attraversamento di un confine tra due stati distinti comporta un onere aggiuntivo di +1 sul costo totale dello spostamento (risultando quindi in un costo totale pari a 2).
- **Penalità Territorio Nemico:** L'ingresso in uno stato ostile applica un malus aggiuntivo istantaneo al costo del tragitto. Tale malus è calcolato moltiplicando il livello di pericolosità del territorio nemico (con valori compresi tra 1 e 9) per un parametro di calibrazione X (da definire in fase di configurazione). Questo approccio parametrico permette all'algoritmo di ricerca di valutare percorsi più o meno prudenti, bilanciando in modo dinamico la brevità del tragitto con l'esposizione al rischio.

Formalizzazione Matematica ed Esempio

Dato un percorso che attraversa n celle, interseca k confini di stato e prevede l'ingresso in m stati nemici (ciascuno caratterizzato da un livello di pericolosità L_i), il costo totale C è espresso dalla seguente formula:

$$C = n + k + \sum_{i=1}^m (L_i \cdot X) \quad (1)$$

Esempio pratico: Uno spostamento attraverso 6 celle all'interno dello stesso stato alleato ha un costo pari a 6. Se lo stesso spostamento prevede l'attraversamento di un confine per entrare in uno stato nemico con livello di pericolosità 4, il calcolo del costo diverrà: 6 (costo celle)+1 (costo confine)+4X (malus nemico). Il costo totale sarà quindi pari a $7 + 4X$.

1.3 Classificazione e Tipologia dei Territori

In fase di acquisizione, ogni cella viene analizzata e classificata per definirne le proprietà fisiche e tattiche. Invece di utilizzare nomenclature estese, il sistema adotta una codifica alfanumerica ottimizzata:

Identificatore	Tipologia	Proprietà
S	Start	Punto di origine dell'agente.
W	Win	Obiettivo finale o destinazione del percorso.
A	Alleato	Territorio amichevole con transito standard.
X	Confine Naturale	Ostacolo insormontabile (non attraversabile).
1 - 9	Territorio Nemico	Rappresenta il grado di pericolosità crescente. I valori numerici sono integrati nella funzione di costo globale.

Tabella 1: Codifica alfanumerica e tipologia dei territori acquisiti

2 Architettura del Software e Scelte Progettuali

L'architettura del sistema è stata sviluppata seguendo un approccio modulare, ispirato ai principi dei microservizi, al fine di garantire scalabilità e facilità di debugging. Inizialmente concepito come uno script monolitico, il progetto è stato rifattorizzato in tre moduli distinti per evitare conflitti tra le librerie di elaborazione delle immagini e quelle di apprendimento automatico. Questa separazione delle responsabilità permette di isolare eventuali errori logici o di classificazione, garantendo che interventi sulla rete neurale non compromettano la gestione geometrica dei confini. La struttura finale si compone di:

- **Modulo Digitalizzatore** (`modulo_digitalizzatore.py`): Specializzato nelle operazioni di pre-elaborazione e computer vision. Prepara l'immagine, ispessisce l'inchiostro e rimuove i quadretti.
- **Modulo ML** (`modulo_ml.py`): Deputato al riconoscimento dei caratteri tramite reti neurali. Prende il ritaglio pulito, lo scala e restituisce il carattere predetto.

- **Hub Centrale (hub_master.py):** Agisce come controller principale del flusso di esecuzione, coordinando i moduli, isolando le isole di pixel, colorando gli stati e assemblando la matrice finale.

3 Sviluppo del Modello ML: Riconoscimento dei Caratteri

Il modulo `modulo_ml.py` funge da wrapper per una Rete Neurale Convolutionale (CNN). Per ottimizzare le prestazioni, il modello viene caricato in memoria esclusivamente all'avvio del programma.

3.1 Creazione del Modello: Dalla MLP alla CNN

Inizialmente, il Machine Learning per riconoscere lettere e numeri scritti a mano era stato strutturato con un'architettura MLP (Multilayer Perceptron). Tuttavia, i risultati non erano soddisfacenti. La criticità risiedeva nel modo in cui l'MLP elabora i dati: richiede un vettore piatto (l'appiattimento di una foto 28×28 in un vettore da 784 elementi), causando la perdita totale della percezione spaziale e geometrica delle lettere. Si è quindi deciso di passare a una CNN (Rete Neurale Convolutionale), dove le immagini vengono analizzate come griglie 2D. Il passaggio ha richiesto due modifiche: applicare la normalizzazione tra 0 e 1 e ridare all'immagine la dimensione 2D.

```
# Normalizzazione tra 0 e 1
X_train = X_train / 255.0
X_test = X_test / 255.0

# Reshape dei dati per renderli compatibili con i layer Conv2D
X_train = X_train.reshape(-1, IMG_SIZE, IMG_SIZE, 1)
X_test = X_test.reshape(-1, IMG_SIZE, IMG_SIZE, 1)
```

La sostituzione del blocco MLP con l'architettura convoluzionale è stata strutturata come segue:

```
# Nuovo Modello CNN
model = keras.Sequential()
# Layer di Input: riceve immagini 28x28 con 1 canale
model.add(layers.Input(shape=(IMG_SIZE, IMG_SIZE, 1)))
# Primo blocco Convoluzionale (estrae feature come bordi e linee)
model.add(layers.Conv2D(32, kernel_size=(3,3), activation="relu"))
model.add(layers.MaxPooling2D(pool_size=(2,2)))
# Secondo blocco Convoluzionale (estrae feature complesse come
# curve e angoli)
model.add(layers.Conv2D(64, kernel_size=(3,3), activation="relu"))
model.add(layers.MaxPooling2D(pool_size=(2,2)))
# Appiattisce i dati prima della classificazione finale
model.add(layers.Flatten())
# Dropout per evitare l'overfitting
model.add(layers.Dropout(0.5))
```

3.2 Il Dataset EMNIST e la necessità del Layer di Ragionamento

Nonostante il passaggio alla CNN, il riconoscimento iniziale presentava delle falle. Il problema derivava dalle caratteristiche del dataset di addestramento EMNIST: le lettere nel dataset riempiono quasi interamente lo spazio, scalate per occupare un quadrato di 20×20 pixel posizionato esattamente al centro di una tela nera di 28×28 pixel. A causa di ciò, una piccola concentrazione di pixel quasi verticale al centro dell'immagine assomigliava statisticamente di più al pattern di un "1" piuttosto che alle ampie curve di una "S" (che il modello si aspettava arrivassero quasi fino ai bordi). Per fornire maggiore capacità di "ragionamento" alla rete, è stato aggiunto un layer denso intermedio prima dell'output:

```
model.add(layers.Dense(128, activation="relu"))
model.add(layers.Dense(NUM_CLASSES, activation="softmax"))
```

3.3 Tentativi di Generalizzazione e Modifiche in Inferenza

Dopo la prima correzione, la precisione era di 3 previsioni corrette e 4 errate su esempi manuali. Per migliorare la generalizzazione sono stati tentati diversi approcci:

- **Data Augmentation:** Si è provato ad aggiungere distorsioni casuali ma leggere (RandomRotation fino al 5%, RandomZoom e RandomTranslation fino al 10%) nel training set. La modifica non ha portato alcun miglioramento allo score ed è stata annullata.
- **Filtro di Sfocatura (Gaussian Blur):** In fase di inferenza si è tentato di "ammorbidire" il tratto per simularne uno simile all'inchiostro del dataset, utilizzando `img.filter(ImageFilter.GaussianBlur(radius=0.8))`. Anche questo tentativo è stato scartato per scarsi risultati.
- **Standardizzazione dell'Input:** La scelta vincente (che ha portato il punteggio a 6 caratteri corretti su 7) è stata processare le immagini di test in inferenza esattamente come nel training set: ritaglio delle parti inutili, centratura della lettera e scalatura mantenendo il rapporto d'aspetto, con lato massimo fissato a 20 pixel all'interno del riquadro 28×28 . Grazie alla precisione geometrica dell'Hub, si è potuto evitare l'uso di librerie pesanti come SciPy per la sola ricerca delle forme.

3.4 Filtri Anti-rumore e l'Integrazione finale di SciPy

Rimaneva un problema di robustezza al rumore: un semplice puntino isolato vicino a un "9" ingannava il sistema, facendogli credere che il carattere iniziasse molto più in alto. Di conseguenza il "9" veniva schiacciato in basso e predetto erroneamente come un "6" sbilenco. Inizialmente si è alzata la soglia di rilevamento dell'inchiostro, ignorando i grigi scuri e richiedendo almeno 2 pixel vicini:

```
# Ricerca intelligente: soglia aumentata a 50
righe_con_pixel = np.sum(arr > 50, axis=1) > 2
colonne_con_pixel = np.sum(arr > 50, axis=0) > 2
```

Poiché l'errore persisteva in alcuni casi limite, si è infine deciso di adottare un approccio più drastico utilizzando la libreria **SciPy** (già disponibile tramite scikit-learn) per rilevare

ed isolare esclusivamente l'isola di pixel più grande associata al carattere, eliminando completamente i puntini di rumore. Questa modifica ha portato il ML a una precisione del 100%.

4 Il Modulo Digitalizzatore: Computer Vision e Pre-elaborazione

Data la natura della mappa (disegnata a mano), si è optato per tecniche di pulizia dell'immagine tramite la libreria OpenCV al posto di addestrare un ML specifico per la digitalizzazione geografica. Il modulo ha il compito di trasformare una fotografia grezza in una matrice binaria pulita.

4.1 Evoluzione del Programma: Dalla Base alle Soglie Adattive

L'idea iniziale prevedeva la conversione in scala di grigi, il rimpicciolimento su griglia 100×100 e una soglia fissa (Threshold). Tuttavia, su fogli a quadretti e senza flash i risultati erano inutilizzabili. Si è passati quindi a una **Soglia Adattiva** (`cv2.adaptiveThreshold`), calcolando il contrasto su finestre locali di 20×20 (poi affinate a 31×31 pixel). Questo rende il programma intelligente nel distinguere sfondi ombreggiati dai tratti reali. A ciò è stato affiancato un **Median Blur** (intensità 11) per cancellare le linee sottili dei quadretti mantenendo intatti i tratti marcati.

4.2 Gestione del Tratto di Penna e Rimozione Quadretti

Un ostacolo critico era l'uso di una penna dal tratto molto sottile, simile allo spessore della griglia del foglio, accentuato dalla riduzione di risoluzione.

1. **Tentativo di Dilatazione:** Si è tentato di usare un pennello 3×3 (`cv2.dilate`) per spalmare l'inchiostro e unire i buchi. Il tratto diveniva perfetto, ma il rumore aumentava drasticamente.
2. **Erosione Morfologica:** Per ottimizzare il sistema salvando i tratti deboli senza conservare i quadretti, si è optato per l'Erosione Morfologica *prima* di sfocare l'immagine. Usando un kernel ridotto, si espandono i pixel neri (ispessendo la penna) permettendo poi di alzare il livello del Median Blur in sicurezza.

```
# Creiamo un pennello 2x2
kernel_penna = np.ones((2,2), np.uint8)
# L'erosione in scala di grigi rende i tratti scuri piu
    spessi
img = cv2.erode(img, kernel_penna, iterations=1)
```

3. **Chiusura dei Buchi:** Infine, è stata applicata la Chiusura Morfologica per saldare perfettamente le discontinuità nel tratto senza ingrassare eccessivamente la linea.

```
# MORPH_CLOSE salda i buchi interni al tratto
kernel_chiusura = np.ones((5,5), np.uint8)
img_binaria = cv2.morphologyEx(img_binaria, cv2.
    MORPH_CLOSE, kernel_chiusura, iterations=2)
```

È importante sottolineare la rimozione della funzione di pulizia globale (precedentemente denominata "Vacuum Cleaner"). Sebbene efficace nel rimuovere artefatti esterni, essa avrebbe eliminato anche i caratteri interni agli stati, fondamentali per l'etichettatura.

5 L'Hub Master: Integrazione e Logica di Sistema

L'Hub Master coordina i moduli e risolve le criticità algoritmiche emerse durante l'unione delle varie parti. Nel primo test completo vi fu un'esplosione di falsi positivi poiché mancava la rimozione del rumore globale.

5.1 Classificazione tramite Analisi delle Aree

Per filtrare gli elementi, è stata introdotta una classificazione basata sulla funzione `connectedComponent`, valutando l'area in pixel delle "isole":

- **Area < 300/400:** Classificata come rumore o "polvere" e scartata.
- **Area compresa tra 300/400 e 15.000:** Identificata come potenziale carattere alfanumerico (in grado di sopravvivere alla pulizia) e inviata al modulo ML.
- **Area > 15.000:** Identificata come confine geografico o stato.

Eventuali falsi positivi residui (come un "7" rilevato nel rumore esterno alla mappa) vengono neutralizzati dalla funzione `cv2.pointPolygonTest`, la quale garantisce che solo i caratteri contenuti *all'interno* di un poligono chiuso vengano considerati nel calcolo dei percorsi.

5.2 Topologia degli Stati e Colorazione

Un problema iniziale riguardava l'uso della funzione `cv2.RETR_EXTERNAL`, che analizzava unicamente il contorno perimetrale globale. Per separare e colorare individualmente gli stati, si è scartato l'uso di algoritmi complessi (come il Teorema dei Quattro Colori) a favore della semplice assegnazione di colori unici da una palette predefinita di 12 toni. La soluzione topologica definitiva ha previsto l'inversione dello spazio di ricerca: invece di concentrarsi sulle linee nere, l'algoritmo utilizza `connectedComponents` per identificare gli spazi bianchi vuoti isolati (usando i confini neri come ostacoli), definendoli in modo perfetto come territori distinti a cui assegnare gli identificativi cromatici.